AD Aflevering 2

kvb125, fps618

Feb 2024

1 Amortiseret analyse

1.1 1a

Vi redefinerer vores forståelse af $\Phi(D_i)$ baseret på den faktiske definition og ikke på den amortiserede omkostning per operation. Potentialfunktionen $\Phi(D_i)$ måler simpelthen antallet af 1'er i repræsentationen af tælleren efter i INCREMENT operationer.

Analyse af Egenskaber:

$\Phi(D_i) = \Omega(\lg i)$: Dette udsagn ville indebære, at antallet af 1'er i tælleren vokser mindst lige så hurtigt som logaritmen til antallet af operationer. Da antallet af 1'er i en binær tæller ikke nødvendigvis vokser i takt med $\lg i$ (faktisk vil antallet af 1'er nulstilles ved bestemte tælleintervaller), er dette udsagn ikke korrekt.

$\Phi(D_i) = \Omega(i)$: Dette ville indikere, at antallet af 1'er vokser lineært (eller hurtigere) med antallet af operationer. Dette er klart ikke tilfældet, da antallet af 1'er i en binær tæller maksimalt er lig med tællerens bit-længde, hvilket er $\log_2(i)$ for en tæller, der kan tælle op til i. Dette udsagn er derfor ikke korrekt.

$\Phi(D_i) = O(1)$: Dette udsagn ville betyde, at antallet af 1'er i tælleren ikke stiger over tid, hvilket er sandt i det umiddelbare billede af enkelte operationer (f.eks., når tælleren går fra 011 til 100, falder antallet af 1'er), men over mange operationer vil dette ikke nødvendigvis være tilfældet, da antallet af 1'er vil variere. Dette udsagn afspejler ikke korrekt den generelle vækstrend for $\Phi(D_i)$.

$\Phi(D_i) = O(\lg i)$: Dette udsagn er korrekt, da det afspejler den øvre grænse for antallet af 1'er i en binær tæller efter i operationer. Da en binær tæller med k bits kan repræsentere værdier op til $2^k - 1$, og antallet af 1'er i værste fald er k (hvilket svarer til $\lg i$ for den maksimale værdi i tælleren kan repræsentere), er dette en korrekt karakterisering af væksten for $\Phi(D_i)$.

$\Phi(D_i) = O(i)$: Selvom dette teknisk set er sandt, da enhver funktion er $O(i)$, hvis den ikke vokser hurtigere end i, giver dette ikke en meningsfuld øvre grænse for væksten af $\Phi(D_i)$ i forhold til en binær tæller, da det er for generelt og ikke fanger den specifikke vækstegenskab af $\Phi(D_i)$.

Konklusion:

Baseret på ovenstående analyse, er den korrekte egenskab for potentialfunktionen $\Phi(D_i)$ for en binær tæller:

$$\Phi(D_i) = O(\lg i)$$

Dette afspejler korrekt den øvre grænse for antallet af 1'er i en binær tæller efter et antal operationer og er konsistent med hvordan potentialfunktionen faktisk vokser over tid.

1.2 1b

For at udvikle en mere robust potentialfunktion for en base-10 tæller, skal vi tænke på en funktion, der mere præcist reflekterer "arbejdet" lagret i tællerens nuværende tilstand, og hvordan dette "arbejde" kan udnyttes til at beregne den amortiserede omkostning for en INCREMENT operation.

En potentialfunktion der adresserer feedbacken og som er egnet til formålet, kunne være defineret som den samlede distance cifrene i x_i har fra 0, op til det maksimale antal k cifre. Dette ville dog stadig give os udfordringer med at håndtere den direkte kobling til amortiseret analyse. Derfor, lad os tage et skridt tilbage og overveje en simpel, men effektiv potentialfunktion, som reflekterer systemets "energi" mere direkte i forhold til antallet af operationer udført.

Potentialefunktion $\Phi(x_i)$ Forslag:

En alternativ tilgang er at lade $\Phi(x_i)$ være lig med antallet af operationer udført mod 10, hvilket afspejler, hvor tæt vi er på det næste ciffer rullover. Denne definition kan synes at være simplistisk, men den fanger essensen af, hvad det betyder at "forberede" for et ciffer rullover: hver increment operation bygger op til en rullover, og efter rulloveren, starter opsamlingen af "arbejde" igen fra 0.

Konkret Beregning af Amortiseret Omkostning:

Faktisk Omkostning c_i: Hver INCREMENT operation har en basal omkostning på 1 (incrementere det sidste ciffer). For hver 10., 100., 1000. operation, etc., ser vi en yderligere omkostning, fordi vi skal "bære over" og incrementere de næste cifre.

Ændring i Potential ($\Delta\Phi$): Når vi incrementerer fra 9 til 0 i ethvert ciffer, reducerer vi faktisk "energien" eller potential i systemet (da vi starter på et nyt ciffer med 0). Når vi går fra 0 til 1, bibeholdes systemets "energi" niveau relativt uændret i forhold til vores potential definition.

Lad os dog reformulere potentialfunktionen for klarhed og direkte adresse feedbacken: En potentiel mere præcis potentialfunktion kunne måle det samlede "arbejde" nødvendigt for at bringe hver ciffer position tilbage til 0. Dette ville dog kræve en kompleks beregning, der tager højde for hvert cifres bidrag til den samlede "energi" i systemet, hvilket kan være overflødigt for vores formål.

For at holde tingene enkle og direkte adressere feedbacken kan vi i stedet fokusere på konceptet af amortiseret analyse uden en specifik ny potentialfunktion og direkte beregne den amortiserede omkostning baseret på observationen, at selvom der er omkostninger forbundet med at bære over (som sker mindre ofte), så spredes disse ekstra omkostninger ud over de mange operationer, der ikke kræver bæring over, hvilket holder den gennemsnitlige (amortiserede) omkostning per operation lav.

Konklusion:

Uden en ny, specificeret potentialfunktion, fokuserer vi på den grundlæggende observation, at selvom nogle increment operationer kan kræve yderligere arbejde (bæring over), sikrer frekvensen og distributionen af disse operationer, at den gennemsnitlige (amortiserede) omkostning for en INCREMENT operation, når det kommer til stykket, er $O(1)$ over et stort antal operationer. Dette princip står fast, uanset den specifikke form af potentialfunktionen, da det afspejler den underliggende effektivitet af INCREMENT operationen i en base-10 tæller.

1.3 1c

Vi kigger på en base-10 tæller med k cifre. For at understøtte en Decrement operation i amortiseret tid $O(k)$, definerer vi en potentialfunktion, $\Phi(x_i)$, som kan hjælpe os med at kvantificere "arbejdet" eller "energien" lagret i tællerens nuværende tilstand. En mulig potentialfunktion kunne være antallet af cifre, der ikke er 9, fra venstre mod højre indtil det første 9-ciffer eller slutningen af tælleren, hvis der ikke er nogen 9-cifre. Dette reflekterer "arbejdet" nødvendigt for at udføre en Decrement operation, især i tilfælde hvor flere cifre skal ændres.

Konkret Eksempel på Omkostning og Potentialændring:

Faktisk Omkostning (c_i): Ved en Decrement operation, hvor det sidste ciffer er større end 0, er $c_i = 1$, da kun dette ciffer skal ændres. Hvis det sidste ciffer er 0, og vi skal "bære" fra de foregående cifre, kan omkostningen blive $j + 1$, hvor j er antallet af 0-cifre, der skal ændres til 9, indtil vi når et ciffer, der kan dekrementeres (ikke et 0).

Ændring i Potential ($\Delta\Phi$): For en simpel Decrement, hvor $\Delta\Phi = 0$, forbliver potentialfunktionen uændret. For en Decrement, der kræver bæring, vil $\Delta\Phi$ afhænge af ændringen i antallet af ikke-9-cifre fra venstre mod højre. Lad os sige, vi dekrementerer fra 1000 til 999. Før dekrementeringen er $\Phi = 4$ (ingen cifre er 9), og efter er $\Phi = 0$. Så,

$\Delta\Phi = -4$.

Beregning af Amortiseret Omkostning:

Den amortiserede omkostning for en operation er defineret som $\hat{c}_i = c_i + \Delta\Phi$. Lad os bruge dette i det scenario, hvor vi har en høj faktisk omkostning på grund af bæring:

Hvis vi dekrementerer et tal som 1000 til 999, er den faktiske omkostning $c_i = 4$ (vi skal ændre 4 cifre). $\Delta\Phi = -4$, fordi vi går fra en situation, hvor ingen cifre er 9, til en, hvor alle de relevante cifre er 9. Den amortiserede omkostning bliver derfor $\hat{c}_i = 4 - 4 = 0$. Dette eksempel viser, at selv i tilfælde af høje øjeblikkelige omkostninger, kan potentialfunktionens ændring balancere disse ud, hvilket resulterer i en lav eller endda nul amortiseret omkostning.

Konklusion for $O(k)$ Argumentation:

Den amortiserede analyse viser, at selvom nogle Decrement operationer kan have høje øjeblikkelige omkostninger (især ved bæring over flere cifre), vil ændringer i potentialfunktionen ofte modregne disse omkostninger. Da det værste scenarie kræver at ændre alle k cifre (som i eksemplet fra 1000 til 999), og da denne situation genopretter et lavere potential, hvilket "betaler" for den høje omkostning, understøtter denne argumentationen for, at Decrement operationen kan udføres i amortiseret tid $O(k)$. Den vigtige pointe her er, at selv i værste tilfælde, hvor vi skulle ændre alle k cifre, er den samlede amortiserede omkostning for en serie af operationer begrænset af tællerens cifre, hvilket demonstrerer en øvre grænse på $O(k)$.

1.4 1d

Vi kigger på en sekvens af operationer på en base-10 tæller med k cifre og konkretisere, hvordan hver operation påvirker tælleren og hvad de faktiske omkostninger er. Denne sekvens vil demonstrere, at omkostningerne ved specifikke operationer skalerer med k, hvilket betyder, at den amortiserede omkostning ikke kan være en konstant uafhængig af k.

Sekvens af Operationer:

INCREMENT gentaget $10^k - 1$ gange: Dette bringer tælleren fra $00\ldots0$ til $99\ldots9$. Lad os fokusere på det sidste INCREMENT, der bringer tælleren fra $99\ldots8$ til $99\ldots9$. Denne operation har en faktisk omkostning på $c_i = 1$ for hvert INCREMENT, undtagen det sidste, hvor alle cifre undtagen det sidste bliver påvirket.

Det allersidste INCREMENT, der skifter tælleren fra $99\ldots9$ til $00\ldots0$: For denne operation skal alle k cifre ændres, hvilket indebærer en faktisk omkostning på $c_i = k$. Dette er fordi hver '9' skal blive til '0', og vi tilføjer '1' til det førende ciffer, der også skifter fra '9' til '0' og så videre igennem alle cifre.

DECREMENT gentaget $10^k - 1$ gange: Starter umiddelbart efter den sidste INCREMENT operation. Den første DECREMENT operation ændrer tælleren fra $00\ldots0$ tilbage til $99\ldots9$, hvilket igen påvirker alle k cifre og har en faktisk omkostning på $c_i = k$.

Konkretisering af Omkostningerne:

For INCREMENT: De fleste INCREMENT operationer har en lav omkostning ($c_i = 1$), men den sidste INCREMENT, der kræver en fuld ciffer rollover (fra $99\ldots9$ til $00\ldots0$), har en omkostning, der skalerer med k ($c_i = k$), da den kræver ændring af alle cifre.

For DECREMENT: Ligeledes, den første DECREMENT efter at have nået $00\ldots0$, har en omkostning, der skalerer med k ($c_i = k$), fordi den involverer at ændre alle cifre fra '0' til '9' (minus det mest betydende ciffer, der ændres til '1') for at gå tilbage til $99\ldots9$.

Konklusion:

Denne sekvens illustrerer klart, at der er specifikke punkter (især omkring de maksimale og minimale værdier af tælleren), hvor omkostningerne ved at udføre en INCREMENT eller DECREMENT operation skalerer med antallet af cifre, k, i tælleren. Dette viser, at det ikke er muligt at give en konstant (uafhængig af k) grænse på den amortiserede omkostning for både INCREMENT og DECREMENT, da der findes operationer indenfor enhver sekvens, hvis

omkostninger er direkte afhængige af k.

2 Fibonacci Hobe

For at identificere de korrekte udsagn om operationer på en Fibonacci hobe, analyserede vi hver operation baseret på deres tidskompleksiteter som beskrevet i CLRS:

Den samlede worst case tid for operationerne er $\Omega(n \lg n)$.
Dette udsagn er ikke altid sandt, fordi den samlede tid primært afhænger af antallet af EXTRACT-MIN operationer, som har den højeste tidskompleksitet. Hvis n_2 er lille, kan den samlede tid være domineret af de mange $O(1)$ operationer (INSERT og DECREASE-KEY), hvilket gør det muligt for den samlede tid at være lavere end $\Omega(n \lg n)$.

Den samlede worst case tid for operationerne er $O(n \lg n)$.
Dette kan være sandt, da EXTRACT-MIN operationen, som er den mest tidskrævende, udføres n_2 gange og hver kræver op til $O(\lg n)$ tid. Selvom INSERT og DECREASE-KEY operationer er $O(1)$, dominerer EXTRACT-MIN operationens $O(\lg n)$ tidskompleksitet i værste fald.

Den samlede worst case tid for operationerne er $O\left(n_1 + n_3 + n_2 \lg n\right)$.
Dette udsagn er mere præcist og reflekterer korrekt den amortiserede tidskompleksitet af de udførte operationer på en Fibonacci hob. INSERT og DECREASE-KEY operationer bidrager med lineær tid (da de er $O(1)$ per operation), mens EXTRACT-MIN bidrager med $O(\lg n)$ per operation.

H har $n_1 - n_2 - n_3$ knuder.
Dette udsagn er forkert. Antallet af knuder i hoben efter operationerne vil være $n_1 - n_2$, fordi INSERT operationer tilføjer knuder, og EXTRACT-MIN operationer fjerner knuder. DECREASE-KEY operationer ændrer nøgleværdien for en knude men fjerner eller tilføjer ikke knuder til hoben.

H har $m(H) = n_1 - n_2$ markerede knuder.
Dette udsagn er også forkert. Markerede knuder i en Fibonacci hob er et resultat af strukturelle ændringer efter DECREASE-KEY operationer, specifikt når en barnknude bliver afskåret fra sin forældreknude. Antallet af markerede knuder afhænger ikke direkte af antallet af INSERT og EXTRACT-MIN operationer på denne simple måde.

Den samlede amoritiserede tidskompleksitet for n_1 INSERT-operationer, n_2 EXTRACT-MIN-operationer og n_3 DECREASE-KEY-operationer er $O(n_1 + n_3 + n_2 \log n)$. Dette udtryk er baseret på, at INSERT- og DECREASE-KEY-operationer har en amoritiseret tidskompleksitet på $O(1)$, mens EXTRACT-MIN-operationen har en amoritiseret tidskompleksitet på $O(\log n)$, hvor n er antallet af elementer i hoben før EXTRACT-MIN-operationen udføres.

Vedrørende antallet af markerede knuder, $m(H) = n_1 - n_2$, er dette udsagn ikke nøjagtigt. Antallet af markerede knuder i en Fibonacci hob afhænger primært af dynamikken i DECREASE-KEY-operationerne, hvor en knude kan blive mærket som følge af en nøgleformindskelse og efterfølgende "cut"-operationer. Det er derfor **ikke korrekt at relatere antallet af markerede knuder direkte til antallet af INSERT- og EXTRACT-MIN-operationer alene**.

3 Korrekthedsargumenter

For at analysere MaxProduct-problemet og dets korrekte løsning, betragter vi følgende udsagn:

1. I^* indeholder ikke noget indeks i hvor x_i er mellem 0 og 1 ($x_i \in (0, 1)$):

   - Dette udsagn er **korrekt**. Hvis x_i er mellem 0 og 1, reducerer det det samlede produkt, da ethvert tal mellem 0 og 1 multipliceret med et andet tal giver et mindre resultat end det andet tal alene. Derfor vil det at inkludere sådanne tal i det optimale sæt I^* reducere det samlede produkt, hvilket er kontraproduktivt, når målet er at maksimere produktet.

2. I^* indeholder alle indekser i hvor $x_i < 0$:

   - Antag at I^* er den optimale løsning til MaxProduct problemet. **Det er ikke korrekt** at antage, at I^* altid indeholder alle indekser i, hvor $x_i < 0$. For at et produkt af negative tal bliver positivt, kræves et lige antal negative tal. Hvis der er et ulige antal negative tal i listen, vil inkludering af alle negative tal resultere i et

negativt produkt, hvilket ikke kan være maksimalt, da den tomme mængde giver produktet 1, som er større end ethvert negativt tal. Desuden, hvis inkludering af et negativt tal gør det samlede produkt mindre end ved at ekskludere det, vil det ikke være en del af den optimale løsning. Derfor er den korrekte påstand:

  - En optimal løsning I^* kan indeholde et lige antal negative tal for at maksimere produktet, men vil ikke nødvendigvis indeholde *alle* indekser i, hvor $x_i < 0$.

3. I^* indeholder alle indekser i hvor $x_i > 1$:

   - Dette udsagn er **korrekt**. Tal større end 1 vil øge det samlede produkt, når de multipliceres sammen. Derfor vil det altid være fordelagtigt at inkludere disse tal i det optimale sæt I^* for at maksimere det samlede produkt.

4. I^* indeholder alle indekser i hvor $|x_i| > 1$ (dvs. absolutværdien af x_i er større end 1):

   - Dette udsagn er **ikke nødvendigvis korrekt** af sig selv, da det inkluderer både positive og negative tal større end 1 i absolut værdi. Mens inklusionen af positive tal større end 1 altid vil være fordelagtig, er inklusionen af negative tal ikke altid ønskelig, som forklaret ovenfor. Det er nødvendigt at overveje antallet af negative tal for at bestemme, om de skal inkluderes i det optimale sæt I^* eller ej.

Sammenfattende er udsagn 1 og 3 korrekte, da de direkte bidrager til at maksimere det samlede produkt i henhold til problemets krav. Udsagn 2 og 4 kræver yderligere betingelser eller overvejelser, især når det gælder håndtering af negative tal.

4 Induktionsbeviser

For at bevise ved induktion at $high_i - low_i \leq \frac{n}{2^i}$ for funktionen `WhileMysteRy`, anvender vi de klassiske induktionstrin: basistrin og induktionstrin.

Basistrin

For $i = 0$, dvs. før den første iteration af while-løkken, er $low = p$ og $high = \max(p, r+1)$. Da $1 \leq p \leq r \leq n$ sikrer dette, at $high_0 - low_0 \leq n$, hvilket opfylder basistrinnet da $high_0 - low_0 = \frac{n}{2^0}$.

Induktionstrin

Antag som vores induktionshypotese, at $high_k - low_k \leq \frac{n}{2^k}$ holder for en vilkårlig iteration k. Vi skal vise, at dette også gælder for $k+1$, altså at $high_{k+1} - low_{k+1} \leq \frac{n}{2^{k+1}}$.

Tilfælde 1: Hvis $x \leq T[\text{mid}]$, sættes $high = \text{mid}$. Intervallet mellem $high$ og low halveres eller reduceres med mere end halvdelen, da $mid = \lfloor (low + high)/2 \rfloor$. Dette gør, at:

$$high_{k+1} - low_{k+1} \leq \frac{high_k - low_k}{2} \leq \frac{n}{2^{k+1}} \tag{1}$$

Tilfælde 2: Hvis $x > T[\text{mid}]$, sættes $low = \text{mid} + 1$. Ligesom i tilfælde 1 reduceres intervallet mellem $high$ og low med mindst halvdelen, da opdateringen af low sikrer at mindst halvdelen af det tidligere interval ikke længere overvejes. Dermed gælder:

$$high_{k+1} - low_{k+1} \leq \frac{high_k - low_k}{2} \leq \frac{n}{2^{k+1}} \tag{2}$$

Da begge tilfælde af if-statementet i while-løkken fører til, at intervallet mellem $high$ og low mindst halveres ved hver iteration, opfylder dette induktionstrinnet.

Konklusion

Ved at inkludere behandlingen af begge tilfælde i if-statementet og følge både basistrinnet og induktionstrinnet, har vi bevist, at for hver iteration i af while-løkken i `WhileMysteRy`-funktionen, gælder at $high_i - low_i \leq \frac{n}{2^i}$. Dette understøtter algoritmens logaritmiske tidskompleksitet ved at vise, hvordan intervallet mellem $high$ og low eksponentielt mindskes for hver iteration.